TatvaSoft

SOFTWARE DEVELOPMENT · GUIDELINE

TVS-DEV-AI-GL-01

AI Development Tools Guidelines

Responsible, effective and controlled use of AI-assisted development tools across software development and testing.

VERSION	EFFECTIVE DATE	PREPARED BY	APPROVED BY
1.0	28.09.2026	Vishal Shah	Shital Patel

HOW TO READ THIS DOCUMENT

Short on time? Read the one-page **Guidelines at a Glance** — it covers the rules everyone must follow.

Every section opens with a **Key Takeaway**, followed by Do / Don't lists or quick-reference tables.

Need more depth? Each section ends with a **Detailed reference** link to the full supporting guide.

Document Control

Document Control Information

Document Title	AI Development Tools Guidelines
Document ID	TVS-DEV-AI-GL-01
Document Version	Version 1.0
Effective Date	28.09.2026

Document Publication History

Document Prepared By	Vishal Shah
Document Reviewed By	Jigar Prajapati
Document Approved By	Shital Patel
Initial Release Version	1.0
Initial Release Date	28.09.2026

Document Revision History

Version No.	Revision Description	Change Author	Change Approver	Revision Date
1.0	Final Release	Vishal Shah	Shital Patel	25.09.2026

Document Definition

This document provides consolidated guidance for the responsible and effective use of AI-assisted development tools across software development and testing activities. It summarises the required practices for prompting, context management, project configuration, model selection, AI-assisted planning and debugging, MCP integrations, data protection, and human review.

Contents

Guidelines at a Glance	The must-follow rules on one page
Purpose & Scope	Why this exists and who it applies to
Prompting — Do's & Don'ts	How to write prompts that get precise results
Context Engineering	Give the AI only what it needs
Mandatory AI Usage Guidelines	Minimum AI-ready setup for every repository
AI Development Tool Setup	Where each config lives in Copilot, Cursor, Codex, Claude Code
AI Model Selection	Match model strength to task risk and complexity
Explore, Plan, Implement Modes	Work by stage, with a check at every stage
Prompt Review Report — Self Driven	*To be published*
Model Context Protocol (MCP)	Integrate external systems safely
Data Privacy & Licensing	Non-negotiable data-protection rules
AI Dev Tools Champions & Onboarding	Who to ask for help

Guidelines at a Glance

The ten rules below summarise this document. Every team member using AI development tools is expected to follow them.

01	**Prompt precisely** State goal, context, scope, constraints and expected outcome. No vague prompts like "Fix this".	Prompting
02	**One task per chat** Start a fresh chat when switching tasks. Reference files — don't paste whole codebases.	Context Engineering
03	**Make every repo AI-ready** Project instruction file (AGENTS.md / CLAUDE.md), named AI champion, sandbox branch, CI/PR review checks.	Mandatory Usage
04	**Configure tools at project level** Start with restrictive permissions; use skills, agents, hooks and MCP deliberately.	Tool Setup
05	**Use the lowest model that works** Escalate to a stronger model only for complex, risky or critical work. Select models explicitly.	Model Selection
06	**Explore → Plan → Implement → Check / Review** A human check closes every stage. Understand and Plan are read-only. Debug only when something fails.	Plan, Ask & Debug
07	**Least privilege for MCP** Read-only where possible, approved servers only, confirm the target before any write action.	MCP
08	**Never share secrets or sensitive data** No credentials, client data, PII or production data in AI tools. Company-approved accounts only.	Data Privacy
09	**Humans own the output** All AI output is reviewed, built and tested before commit, merge or release. It is never trusted by default.	Data Privacy
10	**Report exposure immediately** Accidental data exposure must be reported at once — never concealed or delayed.	Data Privacy

PRECEDENCE

Where client-specific, contractual, regulatory, privacy, security or licensing requirements are **stricter** than these guidelines, the stricter requirement always applies.

Purpose & Scope

Purpose

To provide a common set of guidelines for the **responsible, effective and controlled** use of AI-assisted development tools, so that teams use AI consistently while maintaining:

Software Quality	Security	Traceability	Client & Project Compliance

It defines expected practices for prompt design, context management, project-level AI configuration, model selection, planning and debugging workflows, MCP integrations, data protection, licensing, and human review.

Scope

Aspect	Applies to
Who	Developers, reviewers, technical leads and other team members using AI development tools.
Tools	GitHub Copilot, Cursor, OpenAI Codex, Claude Code, or other approved AI development tools.
Activities	Software development, testing, debugging, code review, documentation, design support and related engineering work.
Topics covered	• Project-level instructions and guardrails • Prompt and context management • AI model selection and Ask / Plan / Debug workflows • Reusable skills and agents; MCP-based integrations • Review and verification of AI-generated output • Handling of sensitive or restricted information

> **PRECEDENCE**
> Client-specific, contractual, regulatory, privacy, security or licensing requirements that are **stricter** than these guidelines take precedence.

Prompting Guidelines — Do's & Don'ts

> **KEY TAKEAWAY**
> A good prompt clearly states the **goal, relevant context, scope, constraints and expected outcome** — and asks for the minimum change needed.

✓ DO

- ✓ Explain the **goal, context, scope, constraints** and **expected outcome**.
- ✓ Specify which files/modules **may change**, what must **stay untouched**, and whether API, database or dependency changes are allowed.
- ✓ Include business rules, current vs. expected behaviour, errors or logs where needed.
- ✓ For complex or unfamiliar tasks, ask the AI to **inspect the existing implementation first**.
- ✓ Break large tasks into **smaller steps** with clear **acceptance criteria**.
- ✓ Review AI changes: only intended files modified, existing behaviour preserved, edge cases and error handling covered.

✗ DON'T

- ✗ Use vague prompts such as **"Fix this"**, **"Make it better"** or **"Optimize this"**.
- ✗ Allow unrelated refactoring or unnecessary rewrites — ask for the **minimum required change**.
- ✗ Reference irrelevant files — keep prompts focused.
- ✗ Continue in the same chat when moving to a different task — **start a new chat**.
- ✗ Include confidential information: **API keys, passwords, connection strings or PII**.

Detailed reference: Prompts - do's and don'ts

Context Engineering

> **KEY TAKEAWAY**
> Give the AI tool **only the information it needs** for the task at hand.

Why it matters

Long conversations, failed attempts, unrelated code or excessive file content make the AI **lose focus** and increase the chance of **incorrect changes**.

✓ DO

- ✓ Start a **fresh chat** when switching tasks or when debugging becomes cluttered.
- ✓ **Reference files directly** instead of repeatedly pasting their contents.
- ✓ Keep stable project rules in **persistent instruction files** or workspace settings.
- ✓ Document architectural and business decisions **in the repository**.
- ✓ Give AI agents **clear boundaries** on what they can modify.

✗ DON'T

- ✗ Carry failed approaches forward into new attempts.
- ✗ Dump large parts of the codebase into the prompt.
- ✗ Rely on chat history as the project's **source of truth**.

Detailed reference: Context Engineering

Mandatory AI Usage Guidelines

> **KEY TAKEAWAY**
>
> Every repository needs a basic **AI-ready setup** before AI tools are used for real development work. AI supports development — it does **not** replace engineering discipline.

Minimum AI-ready setup (per repository)

#	Requirement	Details
1	Project instruction file	A project-level instruction / context file such as `AGENTS.md` or `CLAUDE.md`.
2	Named AI champion	Responsible for keeping the project instructions current.
3	Sandbox / branch	A separate sandbox or branch for low-risk experimentation.
4	Code knowledge graph	For legacy or tightly coupled projects — to identify indirect dependencies and connected changes before code is modified.
5	Team awareness	Developers understand the core concepts: agents, skills, MCP, rules and knowledge graphs.

Repository controls

MANDATORY	• Persistent project instructions • CI / PR checks that enforce review requirements before merge
RECOMMENDED	• Reusable **skills** for repeated tasks • Specialised **agents** for focused work (e.g. review, impact analysis) • **MCP** connections for approved access to Jira, Git, CI or databases • Maintained project documentation for architecture and business decisions • Automated **hooks** where important checks must run consistently (not just written instructions)
OPTIONAL	• Reusable commands • An AI change log recording notable AI-assisted modifications

> **REMEMBER**
>
> AI-generated changes must still be **reviewed, tested and approved** through the existing development and PR process.

Detailed reference: Mandatory AI Usage Guidelines

AI Development Tool Setup

> **KEY TAKEAWAY**
>
> Configure AI tools **at project level** so they follow the same rules, standards, documentation and safety expectations as the team. The four tools use different files, but the **same building blocks**.

Where each building block lives

Area	GitHub Copilot	Cursor	Codex	Claude Code
Project instructions	`.github/copilot-instructions.md`, `.github/instructions/`, `AGENTS.md`	`.cursor/rules/*.mdc`, `AGENTS.md`	`AGENTS.md`, `AGENTS.override.md`	`CLAUDE.md`, `AGENTS.md`, `.claude/rules/`
Guardrails / permissions	VS Code approvals and coding-agent firewall	`.cursor/permissions.json`	`.codex/config.toml`	`.claude/settings.json`
Knowledge graph	Via MCP	Via MCP	Via MCP	Via MCP
Skills	`.github/skills/`	`.cursor/skills/`	`.agents/skills/`	`.claude/skills/`
Agents / subagents	`.github/agents/`	`.cursor/agents/`	`.codex/agents/`	`.claude/agents/`
MCP configuration	`.vscode/mcp.json`	`.cursor/mcp.json`	`.codex/config.toml`	`.mcp.json`
Hooks	`.github/hooks/`	`.cursor/hooks.json`	`.codex/config.toml`, `.codex/hooks.json`	`.claude/settings.json`
Project documentation	`docs/`, README, ADRs	`docs/`, README, ADRs	`docs/`, README, ADRs	`docs/`, README, ADRs
AI change log	`ai-change-log.md`	`ai-change-log.md`	`ai-change-log.md`	`ai-change-log.md`

Setup principles

Building block	What to do
Instructions	Set up from day one: architecture, conventions, build & test commands and known restrictions — so the AI doesn't guess.
Permissions	Start restrictive; expand only where required.
Skills & agents	Skills for repeatable workflows; agents / subagents for focused work — code review, debugging, testing, impact analysis.
MCP	Connect only to approved services: Git/GitHub, Jira, CI, databases, design tools, code-graph services.

Building block	What to do
Hooks	Enforce checks that must not depend on AI memory — run tests, check risky commands, apply approval gates.
Documentation	README, architecture notes and decision records capture decisions not obvious from the code.
AI change log	`ai-change-log.md` gives a simple record of what changed and why for notable AI-assisted changes.

Setup guides: Claude Code Setup Guide | Cursor Setup Guide | GitHub Copilot Setup Guide | Codex Setup Guide

AI Model Selection

> **GOLDEN RULE**
>
> Use the **lowest model tier that can handle the task reliably**. Move to a stronger model only for higher complexity, uncertainty, security concerns, architectural decisions, production data, infrastructure or other high-impact changes.
>
> **Select the model explicitly** — automatic selection may pick a more capable and more expensive model than the task needs.

Choose by task level

Task level	Typical work	Recommended approach
Simple	Small, well-defined changes following an existing pattern.	Lower-tier approved model.
Normal	Feature work across several files or layers within the existing architecture.	Standard development model; create a plan before implementation where appropriate. Testing and review ideally require higher model but if not possible then same model will work
Complex / Critical	Cross-cutting design, authentication / authorisation, production data, infrastructure, migrations, performance-sensitive work.	Stronger approved model + additional review + fresh context + mandatory human approval before merge or deployment.
Testing / Code Review	Verifying and reviewing the implementation.	Same or stronger model as development. For normal and complex work, use a separate agent or fresh context so the review is independent.

Cost & safety

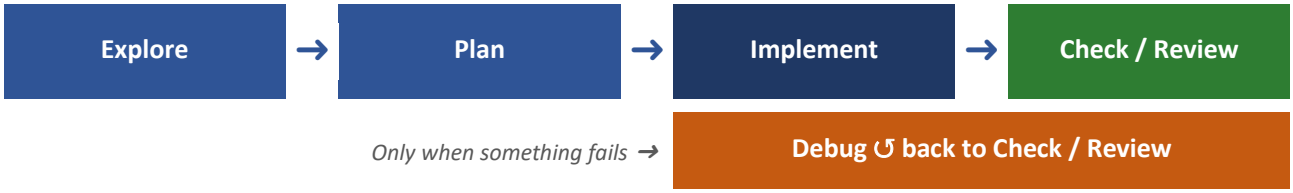
- Reserve stronger models for work that genuinely needs them.
- Use **scripts or automation** for repetitive, deterministic activities where practical.
- Never give secrets, production credentials or restricted information to an AI tool unless the Company or Client has **explicitly approved** that environment.

Detailed reference: AI Model Selection

Explore, Plan, Implement Modes

> **KEY TAKEAWAY**
>
> Work by stage: **Explore → Plan → Implement → Check / Review**, with a human check at each stage. Every AI-generated change remains **the developer's responsibility**.

Typical workflow



Simple tasks can skip Plan. Tool-specific steps (Claude Code, Cursor, GitHub Copilot, OpenAI Codex): TVS-DEV-AI-GL-01.6.

Which stage, when?

Stage	Use it for	Check before moving on
Explore / Understand	Exploring options (**read-only**).	Same understanding of the task
Plan	Approach, steps, edge cases, tests (**read-only**).	**Human approves the plan**
Implement	Carrying out the approved plan in small steps.	Developer reads each diff
Check / Review	Confirming the change is correct and safe.	Build, tests and code review pass
Debug (if needed)	Errors or wrong behaviour, from evidence.	Root cause fixed, test passes

Debugging rules

- Use Debug **only when something fails**; fix at the **failure point**, no unrelated refactoring.
- Add a **regression test** where appropriate; remove temporary logs and breakpoints **before commit**.

Rules for all stages

- Provide precise context; keep **one task per chat**; use project instruction files for persistent rules; select an appropriate model.
- Review, build and test every AI-generated change **before commit**.
- Keep sensitive data, secrets, production credentials and restricted information **out of AI conversations**.
- **Human approval is mandatory** for security-sensitive, production, infrastructure and other high-impact changes.

Detailed reference: AI Modes

Prompt Review Report — Self Driven

> **STATUS**
> Content for this section will be published in a future revision of this document.

Model Context Protocol (MCP) — Integration & Safe Usage

> **KEY TAKEAWAY**
> MCP lets AI tools connect to **approved** external systems. Always apply the **principle of least privilege**.

What MCP is & why use it

A standard way for AI development tools to retrieve project information or perform supported actions through connected services — such as **Figma, Jira, documentation systems** or other internal tools.

Less context switching	Better project context for the AI	Automates repetitive work via natural language

✓ DO

- ✓ Grant **only the permissions required** for the task.
- ✓ Use **read-only access** where possible.
- ✓ Prefer **development / test credentials** over sensitive production accounts.
- ✓ Use MCP servers from **trusted and approved** sources only.
- ✓ Review permissions when a server is **installed or updated**.
- ✓ **Disable or remove** connections no longer required.

✗ DON'T

- ✗ Enable unnecessary **edit, delete or administrative** capabilities.
- ✗ Place API keys, tokens, passwords or other credentials in **source code**, commit them to **Git**, or paste them into **AI conversations**.
- ✗ Allow destructive actions without **reviewing them first**.

Before any write or high-impact action

- Confirm the AI is acting on the **correct project, file, issue or environment**.
- Clearly identify the **intended target and fields**, and review them before the AI performs the update.

Example workflows

- **Figma** → design information assists frontend implementation.
- **Jira** → issue details retrieved as development context (updates only where permitted and reviewed).

Detailed reference: MCP Integration & Safe Usage

Data Privacy & Licensing

NON-NEGOTIABLE

These rules protect TatvaSoft and client data. They are **mandatory** for everyone using AI tools on project work.

Strict Tool Configuration

- Configure all approved AI tools with the **most restrictive security, privacy, data-retention and data-sharing settings** approved by TatvaSoft and/or the client.
- Where available, ensure code, prompts, files and other inputs are **not used for model training** or shared beyond the authorised service scope.

No Unauthorised Setting Changes

- Do **not** alter, downgrade, disable or bypass approved security, privacy, data-sharing, retention or access-control settings.
- Any change requires **prior formal approval** from the PM/PRM and, where applicable, the Information Security team and/or client.

Strictly Prohibited Inputs

Never submit, upload, paste or otherwise expose the following to AI tools — unless explicitly authorised under an approved process:

- ✗ Production secrets, passwords, API keys, tokens, certificates or private keys
- ✗ Customer / client confidential information
- ✗ Personally identifiable information (PII) or other personal data
- ✗ Proprietary or commercially sensitive information
- ✗ Production database content or live production data
- ✗ Database dumps, backups or production data extracts
- ✗ Confidential business documents or credentials
- ✗ Any information restricted by contractual, regulatory or client-specific requirements

When in doubt, don't submit it — confirm its classification and permitted usage first.

No Personal or Unapproved Accounts / Tools

- Do **not** use personal accounts, free/personal-tier accounts or unapproved AI tools for any work involving TatvaSoft or client information (development, testing, design, documentation, etc.).
- Use **only company-approved accounts and AI tools** for project activities.

Data Minimisation

Share **only the minimum information necessary** for the task. Where practical:

- Remove or anonymise sensitive information before submission.
- Use representative or **synthetic data** instead of real data.
- Share only the relevant code snippet / file — not the entire repository or project.
- Remove secrets, credentials, tokens, comments containing confidential information and unnecessary business logic.

> **CRITICAL**
>
> **Do NOT use any AI tool to anonymise data in the first place.**

Code Review & Verification

All AI-generated or AI-assisted output must undergo **human review and verification** before it is committed, merged, deployed or released. Review, as applicable:

- ☐ Functional correctness and fitness for purpose
- ☐ Security vulnerabilities
- ☐ Data protection and privacy implications
- ☐ Credentials, secrets and sensitive information
- ☐ Licensing and open-source compliance
- ☐ Compliance with project / client requirements
- ☐ Unintended data exposure or insecure implementation

*AI output is **never** trusted or production-ready just because an approved tool generated it.*

Incident Response

- **Report immediately** — through the applicable security/incident reporting process — any accidental exposure of personal data, client data, proprietary information, credentials, secrets or other restricted information to an AI tool.
- Do **not** conceal, independently resolve or delay reporting an accidental exposure.

> **NOTE — CLIENT REQUIREMENTS COME FIRST**
>
> Client-specific privacy and security requirements always take precedence over these guidelines. Where a client contract, security policy, DPA, NDA or other documented requirement imposes **stricter** restrictions on AI tool usage, the stricter requirement must be followed.

AI Dev Tools Champions & Onboarding Support

> **KEY TAKEAWAY**
> Every PRM and every project has named **AI Dev Tools points of contact** who are well-versed in these guidelines and experienced with AI Dev Tools and their best practices.

	PRM Level	Project Level
Minimum	At least 2 team members per PRM	At least 1–2 team members per project
Role	AI Dev Tools point of contact within the PRM — guides, assists and supports team members on these guidelines and appropriate AI Dev Tools use.	Project-level AI Dev Tools point of contact — guides, assists and supports project team members.
Focus areas	• AI Dev Tools onboarding & best practices • Onboarding of **new team members**, especially when joining a new project or at project initiation • Ensuring team members understand the applicable AI Dev Tools, security requirements and best practices	• Applicable AI Dev Tools and their approved usage • Project-specific guidelines and restrictions • Security, privacy and data-handling requirements • Recommended AI-assisted development practices • Onboarding new team members to AI Dev Tools • Understanding and following **client- or project-specific** AI requirements

— End of Document —